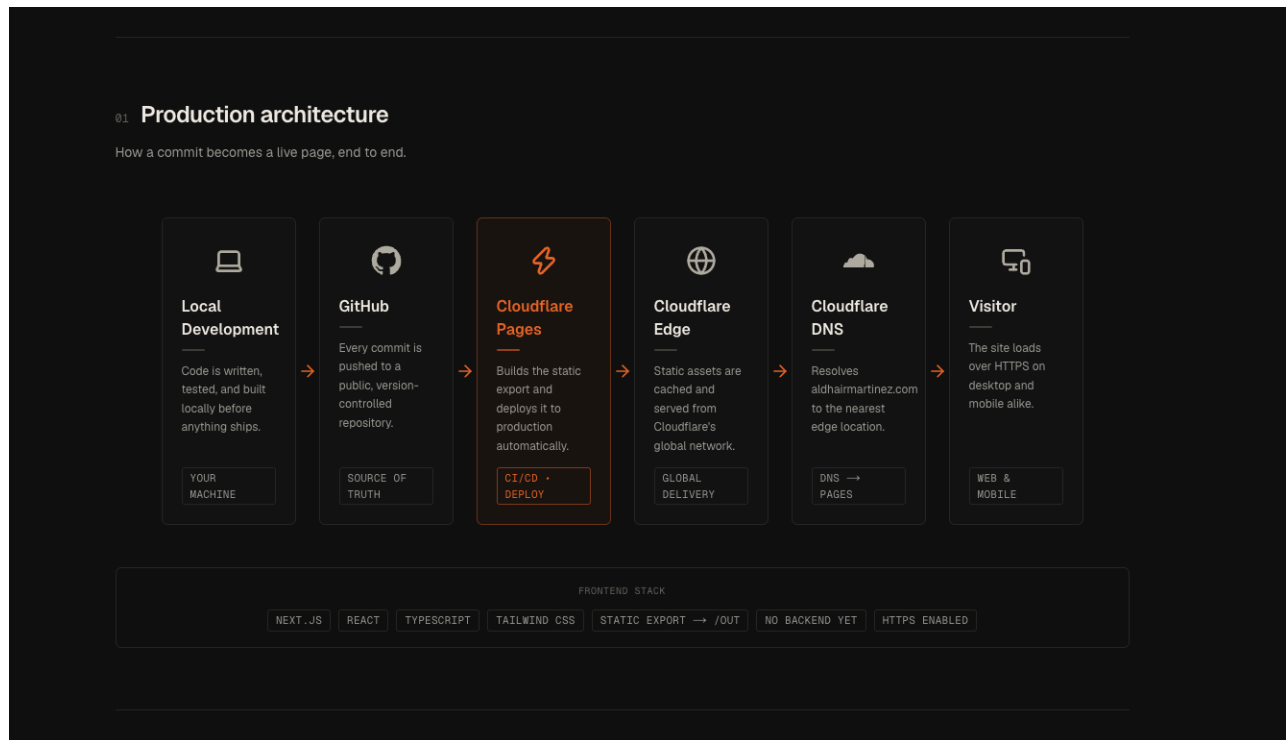


Building, shipping, and observing aldhairmartinez.com

A practical record of the first production version: static Next.js, Git/GitHub, Cloudflare Pages, DNS/HTTPS, environment configuration, and Grafana Faro.

V1 DEFINITION A frontend-only static site that is built, deployed to a real domain, instrumented with Faro, and verified end-to-end in Grafana Cloud.



Production architecture implemented in /observability during V1.

Build → Deploy → Observe → Document → Expand

This is the development rhythm to carry into V2 and beyond.

01

V1 at a glance

Concept	In this project
Application	Next.js + React + TypeScript + Tailwind CSS
Architecture	Static frontend; no backend server yet
Source control	Local Git repository + public GitHub repository
Production hosting	Cloudflare Pages
Domain	aldhairmartinez.com with Cloudflare DNS + HTTPS
Observability	Grafana Faro Web SDK → Grafana Cloud Frontend Observability
Environment tags	localhost = development; live domain = production
Deferred	Session Replay and distributed tracing

KEY MENTAL MODEL The website does not need Python to be online. Next.js builds files ahead of time; Cloudflare serves those files; the visitor's browser renders the UI and runs browser-side JavaScript.

02

How a website can run without a backend

The easiest way to understand V1 is to separate build time, hosting, and browser runtime.

```
SOURCE CODE
  ↓ npm run build
STATIC HTML / CSS / JAVASCRIPT (/out)
  ↓ deploy
CLOUDFLARE PAGES / EDGE
  ↓ request
VISITOR BROWSER
```

Concept	In this project
Build time	Next.js turns the source project into static assets. This happens before a visitor arrives.
Hosting	Cloudflare stores and serves the built assets from its edge network.
Browser runtime	The visitor receives HTML/CSS/JavaScript. Browser-side JavaScript

can still execute even though no Python/FastAPI server exists.

Backend

A server/API that performs request-time logic, database access, authentication, processing, etc. V1 does not have one.

When V2 adds FastAPI, the request path can become browser → API → Python/FastAPI → data/services → response. That will be a real architectural change, not just another library.

03

Git and GitHub: local history vs remote collaboration

Git and GitHub are related but not the same thing. Git is the version-control system on your machine. GitHub hosts a remote copy of the repository and provides collaboration/integration around it.

Concept	In this project
Working tree	The files currently on your laptop, including uncommitted edits.
Staging	The set of changes selected for the next commit.
Commit	A named snapshot in local Git history.
main	The production branch used by this project.
origin	The conventional name for the GitHub remote.
push	Send local commits to GitHub.
pull	Bring remote changes back to the local repository.
.gitignore	Rules that prevent local/generated/sensitive files from being tracked.

```
git status
git add <files>
git commit -m "Describe the change"
git push origin main
```

PRODUCTION CONSEQUENCE A local edit does nothing to production. A local commit still does nothing to production. In this project, a push to GitHub main is what gives Cloudflare a new source revision to build and deploy.

The next maturity step is feature branch → local test → commit → push → preview deployment → review → merge → production. That gives you a safe review boundary before main changes.

04

Cloudflare Pages, CI/CD, DNS, and HTTPS

Cloudflare Pages is the production delivery system for V1. It is connected to the GitHub repository and knows how to build the static Next.js site.

```
GitHub main
  ↓ push detected
Cloudflare build
  ↓ Next.js static export
/out
  ↓ deploy
Cloudflare Pages / edge
  ↓ DNS + HTTPS
aldhairmartinez.com
```

Why Pages, not Workers?

The project initially landed in a Workers-oriented flow that asked for a Wrangler deploy command. That was the clue it was the wrong deployment model. V1 produces a folder of static assets, so Pages is the natural fit. Workers are useful when you need request-time code at Cloudflare's edge.

What CI/CD means here

The useful simplified definition is: a source change in GitHub triggers an automated build and deployment. The important part is not the acronym; it is that production is reproducible from source control instead of manually uploading files.

DNS and HTTPS

- DNS maps the human-readable domain aldhairmartinez.com to the Cloudflare-hosted site.
- The custom domain was activated through Cloudflare Pages using a DNS record that points the domain at the Pages project.
- HTTPS/SSL encrypts traffic between the visitor and the site. Cloudflare shows SSL enabled for the custom domain.

05

Environment variables: local vs production

Environment variables let the same code behave differently depending on where it is built or run, without hardcoding environment-specific values into source files.

Concept	In this project
<code>.env.example</code>	A committed template that documents variable names. It should contain placeholders, not real environment-specific values.
<code>.env.local</code>	Local machine values. Gitignored. Used by the local Next.js dev server.
Cloudflare variables	Production values configured in the Cloudflare Pages project and

available during the production build.

NEXT_PUBLIC_*

Next.js convention for values intentionally made available to browser-side code. These are not secrets.

```
NEXT_PUBLIC_FARO_URL=<collector URL>
NEXT_PUBLIC_FARO_APP_NAME=aldhairmartinez-com
NEXT_PUBLIC_FARO_ENVIRONMENT=development # local
NEXT_PUBLIC_FARO_ENVIRONMENT=production # Cloudflare
```

IMPORTANT NEXT_PUBLIC_FARO_URL is browser-visible by design. Treating a NEXT_PUBLIC_* value as a secret would be a security mistake. A real credential such as an Access Policy token must never be exposed in frontend JavaScript.

Because this is a static export, production NEXT_PUBLIC_* values are baked into the frontend during the Cloudflare build. Changing the Cloudflare variable requires a new build/deployment for the generated site to contain the new value.

06

Grafana Faro: observing the browser

V1 observability is intentionally frontend-only. The Faro Web SDK runs in the visitor's browser and sends browser telemetry to the Grafana Cloud Frontend Observability collector.

```
Browser (localhost or aldhairmartinez.com)
  ↓ Faro Web SDK
Grafana Cloud collector
  ↓
Frontend Observability
```

Concept	In this project
App	aldhairmartinez-com
Allowed origins	https://aldhairmartinez.com and http://localhost:3000
Local tag	environment=development
Production tag	environment=production
Sampling	100% Faro session sampling for V1
Signals	Page loads/navigation, frontend errors, Web Vitals, browser/resource performance, sessions

Why no Access Policy?

The browser ingestion path uses the public collector URL created for the Frontend Observability app. V1 did not need a Grafana Cloud Access Policy token for browser telemetry ingestion. An Access Policy would be relevant for authenticated management/API automation, not for exposing a secret in the browser.

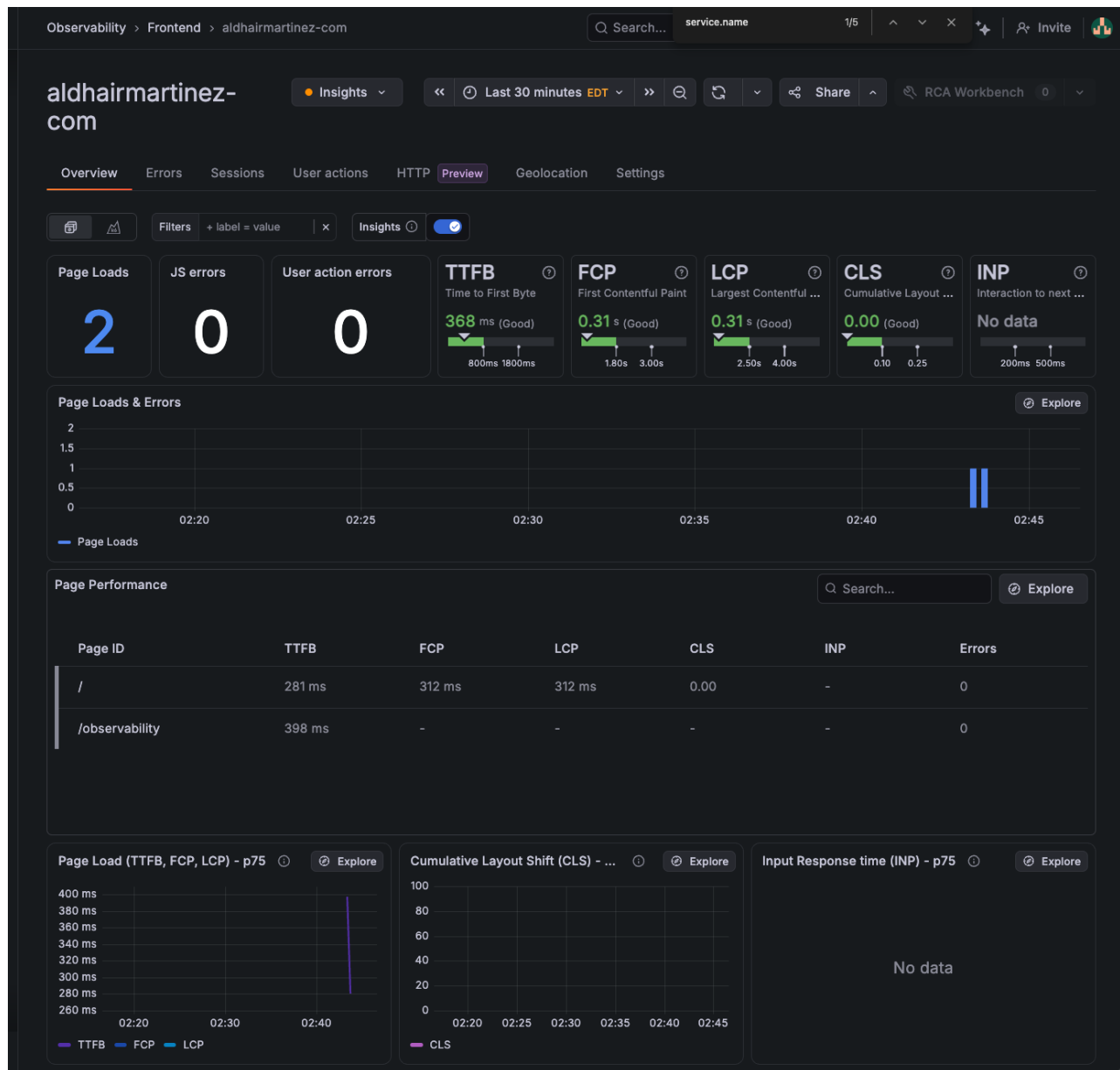
Why CORS matters

CORS (Cross-Origin Resource Sharing) is the browser security mechanism that decides whether code loaded from one origin is allowed to send requests to another origin. The Frontend Observability app explicitly allows the production site and localhost development origin.

What we proved end-to-end

```
localhost:3000
  ↓ OPTIONS preflight
204 No Content
  ↓ telemetry POST
202 Accepted
  ↓
Grafana Cloud data visible
```

WHY THIS MATTERS We did not stop at “the code looks right.” We verified actual network traffic and then verified the resulting data in Grafana Cloud. That is a much stronger validation pattern.



Grafana Cloud Frontend Observability receiving V1 browser telemetry during validation.

The first tiny sample showed page loads for / and /observability plus Core Web Vital measurements. Those initial numbers prove ingestion; they are not enough traffic to make meaningful performance conclusions.

Session Replay and tracing

Session Replay was evaluated but deferred from V1. Distributed tracing was also deliberately deferred until V2 introduces a backend, where a browser trace has a server-side request to continue into.

07

Troubleshooting: the mistakes were part of the lesson

1. Workers instead of Pages

Concept

In this project

Problem	The setup flow asked for a Wrangler deploy command.
Cause	V1 is a static export, so the deployment model did not match the artifact.
Fix	Switched to Cloudflare Pages and configured the Next.js static export build/output.
Lesson	Use the shape of the build artifact to choose the hosting model.

2. GitHub auth mismatch

Concept	In this project
Problem	The GitHub CLI was authenticated to an older/different account identity.
Cause	Local CLI authentication did not match the intended GitHub namespace.
Fix	Reauthenticated gh and verified the active account before creating/pushing the repo.
Lesson	Authentication state is separate from repository state; verify both.

3. Collector URL in .env.example

Concept	In this project
Problem	A real Faro collector URL was temporarily put in the tracked example file.
Cause	The example/template file was confused with the local environment file.
Fix	Moved the real value to gitignored .env.local before committing.
Lesson	Templates document variable names; local environment files hold local values.

4. Local Faro worked, production did not

Concept	In this project
Problem	Grafana showed development telemetry, but the live site still sent nothing.

Cause	Cloudflare had no production NEXT_PUBLIC_FARO_* variables, so the production build was inert.
Fix	Added the production variables in Cloudflare and rebuilt/deployed.
Lesson	Development success does not imply production configuration exists.

5. CORS uncertainty

Concept	In this project
Problem	The code was configured, but it was not yet proven that the browser could send to the collector.
Cause	Cross-origin browser requests require the origin to be allowed.
Fix	Verified OPTIONS → 204 and POST → 202.
Lesson	Inspect the network path, not just application code.

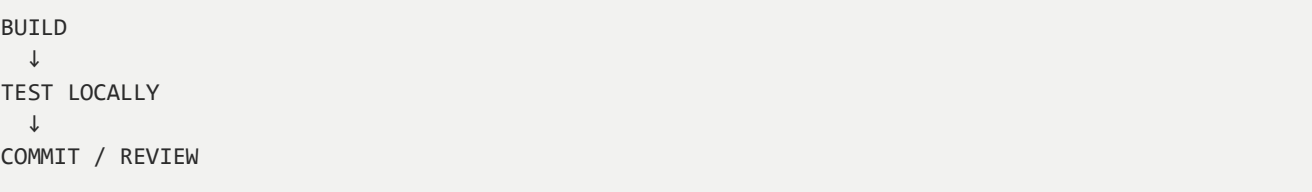
6. Architecture UI clipping

Concept	In this project
Problem	Lint/build passed while visual elements still clipped or aligned awkwardly.
Cause	Build correctness does not validate layout at real viewport sizes.
Fix	Iterated on overflow, wrapping, icon geometry, and alignment using rendered screenshots.
Lesson	Visual QA is a separate test category from compile/lint checks.

08

The engineering workflow to carry forward

V1 established a useful pattern. Each future version should be treated as a small production system rather than a pile of features.



↓
DEPLOY
↓
OBSERVE
↓
VERIFY
↓
DOCUMENT
↓
EXPAND

For V2, the major conceptual jump is adding a backend. That is where FastAPI, server-side OpenTelemetry, request traces, and eventually data services such as PostgreSQL, Redis, and Kafka can become real rather than planned labels.

RULE FOR THE LAB Instrument after each version. That way every new layer teaches both how it works and how it fails.

09

Command + concept cheat sheet

Concept	In this project
<code>npm install</code>	Install dependencies declared by the project.
<code>npm run dev</code>	Start the local Next.js development server.
<code>npm run lint</code>	Run static code-quality checks.
<code>npm run build</code>	Create the production/static build.
<code>git status</code>	See changed/staged/untracked files.
<code>git add</code>	Stage changes for the next commit.
<code>git commit</code>	Create a local versioned snapshot.
<code>git push</code>	Send commits to GitHub; on main this can trigger Cloudflare production deployment.

Glossary

Concept	In this project
Frontend	Code and UI delivered to and executed by the user's browser.

Backend	Server-side code that handles request-time logic and services.
Static export	Prebuilt HTML/CSS/JS files that can be served without a traditional application server.
CDN / edge	Distributed infrastructure that serves content closer to users.
DNS	The naming system that maps domains to the service that should answer for them.
HTTPS	Encrypted HTTP using TLS.
CI/CD	Automated build/test/deployment workflow tied to source changes.
CORS	Browser rules controlling cross-origin requests.
Telemetry	Signals emitted by software so behavior/performance/failures can be observed.
Web Vitals	Browser experience metrics such as LCP, CLS, and INP.

10

What V1 taught

- A website can be fully production-ready without a Python/backend server when its content can be built statically.
- Git is local version control; GitHub is the remote hosting/collaboration layer.
- A push to main can become a production event when a deployment platform is integrated with the repository.
- Cloudflare Pages fits static build artifacts; Workers are for request-time edge code.
- DNS and HTTPS are separate layers from the application code but are part of the production system.
- Environment configuration must exist separately in local development and production.
- NEXT_PUBLIC_* values are intentionally browser-visible and therefore cannot be treated as secrets.
- CORS is a browser security boundary that matters when frontend code sends telemetry to another origin.
- Observability is not “installed” until the telemetry path is verified end-to-end.
- Lint/build success and visual correctness are different tests.
- The best learning loop for this project is build → deploy → observe → document → expand.

V1 complete: static frontend live at aldhairmartinez.com; production delivery automated through GitHub and Cloudflare Pages; browser telemetry verified in Grafana Cloud Frontend Observability.

Prepared as the V1 learning record for aldhairmartinez.com • August 2026